

Dynamic Word Template Filler

Executive Summary

A web-based application that enables users to upload Word document templates using simple placeholder syntax and dynamically generate filled documents through an intuitive form interface.

This Proof of Concept demonstrates a streamlined approach to document generation that eliminates the technical barriers typically associated with template-based systems, making it accessible to business users without programming knowledge.

Business Need

Organizations frequently need to generate customized Word documents from templates including:

- **Contracts** - Client agreements with variable terms and conditions
- **Invoices** - Billing documents with line items and calculations
- **Reports** - Standardized reports with dynamic data sections
- **Letters** - Correspondence with personalized content

Current solutions require technical knowledge of complex templating systems such as Jinja2, Mustache, or mail merge configurations. This POC demonstrates a user-friendly approach using intuitive placeholder syntax that business users can create and maintain independently.

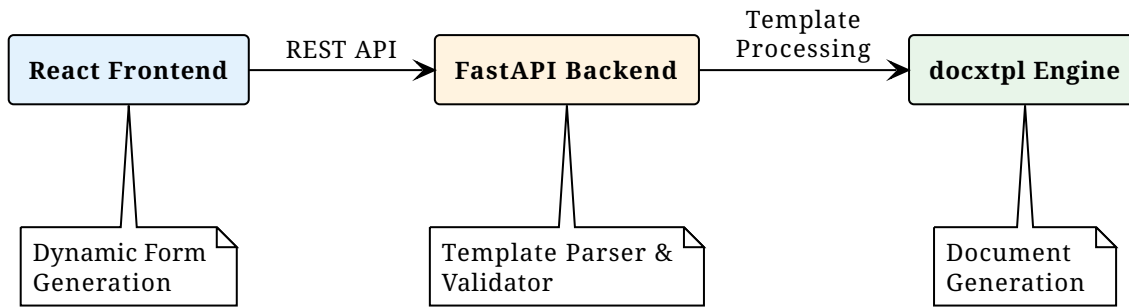
Template Syntax Support

The application supports three primary placeholder types that cover most document generation requirements:

Syntax	Purpose	Example
<code>{field}</code>	Simple text placeholder	<code>{name}, {email}, {company}</code>
<code>{#section}...{/section}</code>	Conditional content blocks	<code>{#disclaimer}Legal disclaimer text here{/disclaimer}</code>
<code>{#list}...{/list}</code>	Repeatable sections for collections	<code>{#products}{name} - \${price}{/products}</code>

System Architecture

The application follows a clean three-tier architecture with clear separation of concerns:



Data Flow:

1. User uploads a **.docx** template via the React frontend
2. Backend parses the template and extracts all placeholder definitions
3. Frontend dynamically generates appropriate form controls
4. User fills in the form data and submits
5. Backend processes the template with user data via docxtpl
6. Generated document is returned for download

User Interface Wireframe

DocxTpl Template Filler

Template Upload

Drag & Drop your .docx template here

or click to browse

Upload Template

Simple Fields

Name: John Smith

Email: john@example.com

Company: Acme Corporation

Optional Sections

☒ Include Disclaimer

☐ Include Terms & Conditions

☒ Include Signature Block

Products (Repeatable)

Product Name	Price	
Widget Pro	\$99.99	Remove
Service Package	\$249.00	Remove

+ Add Product

Generate Document

The interface provides distinct sections for each placeholder type:

- **Simple Fields** - Standard text inputs for basic placeholders

- **Optional Sections** - Checkboxes for conditional content blocks
- **Repeatable Sections** - Dynamic forms with add/remove capabilities

Key Features

Feature	Description
Drag & Drop Upload	Simple template upload with file type validation and size limits
Auto-Detection	Automatically extracts fields, conditionals, and lists from uploaded templates
Dynamic Forms	Generates appropriate input controls based on detected field types
Repeatable Sections	Add/remove items for list-type content with intuitive controls
Conditional Sections	Toggle optional content via checkboxes for cleaner document output
Instant Generation	Download filled document immediately after form submission

Technical Stack

Component	Technology
Frontend	React + TypeScript + Vite
Backend	Python + FastAPI
Template Engine	python-docx-template (docxtpl)
Document Format	OOXML (.docx)

Technology Rationale:

- **React + TypeScript** - Type safety and component reusability for dynamic form generation
- **FastAPI** - High-performance async Python framework with automatic API documentation
- **docxtpl** - Mature Jinja2-based Word templating with excellent format preservation

Success Criteria

The POC will be considered successful when it demonstrates the following capabilities:

- ☐ Parse templates containing `{field}`, `{#conditional}`, and `{#list}` syntax
- ☐ Generate dynamic form UI components from parsed template metadata
- ☐ Support add/remove functionality for repeatable list sections
- ☐ Generate filled documents while preserving original formatting and styles
- ☐ Handle empty and optional fields gracefully without document errors
- ☐ Provide clear error messages for invalid template syntax

❑ Complete document generation in under 5 seconds for typical templates

Timeline Estimate

Phase	Duration
Backend Parser & Generator	2 days
Frontend Dynamic Form	2 days
Integration & Testing	1 day
Total	5 days

Phase Details:

- **Backend Parser & Generator** - Template parsing logic, field extraction, document generation endpoint
- **Frontend Dynamic Form** - Dynamic component rendering, form state management, file upload handling
- **Integration & Testing** - End-to-end testing, error handling, edge case validation

Next Steps

Upon POC approval, the following actions will initiate the development:

1. Set up development environment and project repositories
2. Create sample templates for testing various placeholder types
3. Implement backend template parsing and validation
4. Build frontend dynamic form generation system
5. Integrate components and perform comprehensive testing
6. Document API endpoints and user guide

Document generated: 2026-01-15 | Version: 1.0